

Hardware Secured IoT Gateway

ECSE 4660/6660 Internetworking of Things

05/01/2026

Edgar Muniz, munize@rpi.edu, ECSE 6660

Hayden Fuller, fulleh@rpi.edu, ECSE 6660

1. Objective

This project will develop a Hardware-Secured IoT Gateway and demonstrate a cryptographically verified telemetry pipeline for critical infrastructure. The system will help utility operators by ensuring that sensor readings cannot be spoofed at the software level. This demonstration will benefit society by providing a blueprint for low-power, low-cost, “Zero Trust” IoT gateways where the security keys are physically isolated from the internet facing processor.

2. Innovation and Literature Review

This project provides a new look at IoT security by establishing a reconfigurable, physical trust boundary between the network-facing gateway and the cryptographic core. While traditional IoT security relies on software-level encryption, this design moves the root of trust to physical hardware, efficiently offloading expensive algorithms from sensor nodes with limited resources and providing cycle-accurate determinism.

This work builds upon and differentiates itself from key research in the field:

- Recent research has explored the development of hybrid hardware-software encryptors designed for post-quantum resistance, combining multiple security methods to create highly resilient keys. While those works focus on algorithm strength against future quantum threats, this project will focus on the physical partitioning of trust.
- Existing literature has demonstrated the benefits of offloading heavy digital signature generation from resource-constrained sensor nodes to a nearby gateway accelerated by a GPU. However, this project utilizes an FPGA instead of a GPU. FPGAs provide a cycle-accurate determinism which is essential for real time fault response.

<https://www.sciencedirect.com/science/article/pii/S2542660525000678>

P. Cíbik, S. Ricci, Patrik Dobiáš, J. Hajný, L. Malina, and J. Havlín, “Quantum-resistant hardware-accelerated IoT traffic encryptor,” *Internet of Things*, pp. 101554–101554, Mar. 2025, doi: <https://doi.org/10.1016/j.iot.2025.101554>.

<https://ieeexplore.ieee.org/abstract/document/8536371>

C. -C. Chang, W. -K. Lee, Y. Liu, B. -M. Goi and R. C. . -W. Phan, "Signature Gateway: Offloading Signature Generation to IoT Gateway Accelerated by GPU," in *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4448-4461, June 2019, doi: 10.1109/JIOT.2018.2881425.

3. Methodology

We simulate a real world situation by monitoring sensor data via MQTT.

- We use a DHT11 to collect sensor data
- The Raspberry Pi reads this raw data from the DHT11 via the proprietary DHT protocol
- The RPi compiles the data (could be from multiple sensors) into a 128-bit payload
- The RPi generates a unique AES-CTR counter block with a nonce and incrementing counter
- The RPi sends the counter block to the Nexys A7 (FPGA) via an SPI bus.
- The FPGA's AES-128 engine generates a keystream using a key stored in its logic
- The FPGA send this keystream back to the RPi via the same SPI bus
- The RPi XORs the keystream with the sensor data to produce ciphertext
- The ciphertext is packaged as a JSON and published to the MQTT broker
- The PC running Node-RED is subscribed to receive the packet
- Node-RED decrypts the data in software
- Node-RED reads the data from the payload and displays it on the dashboard

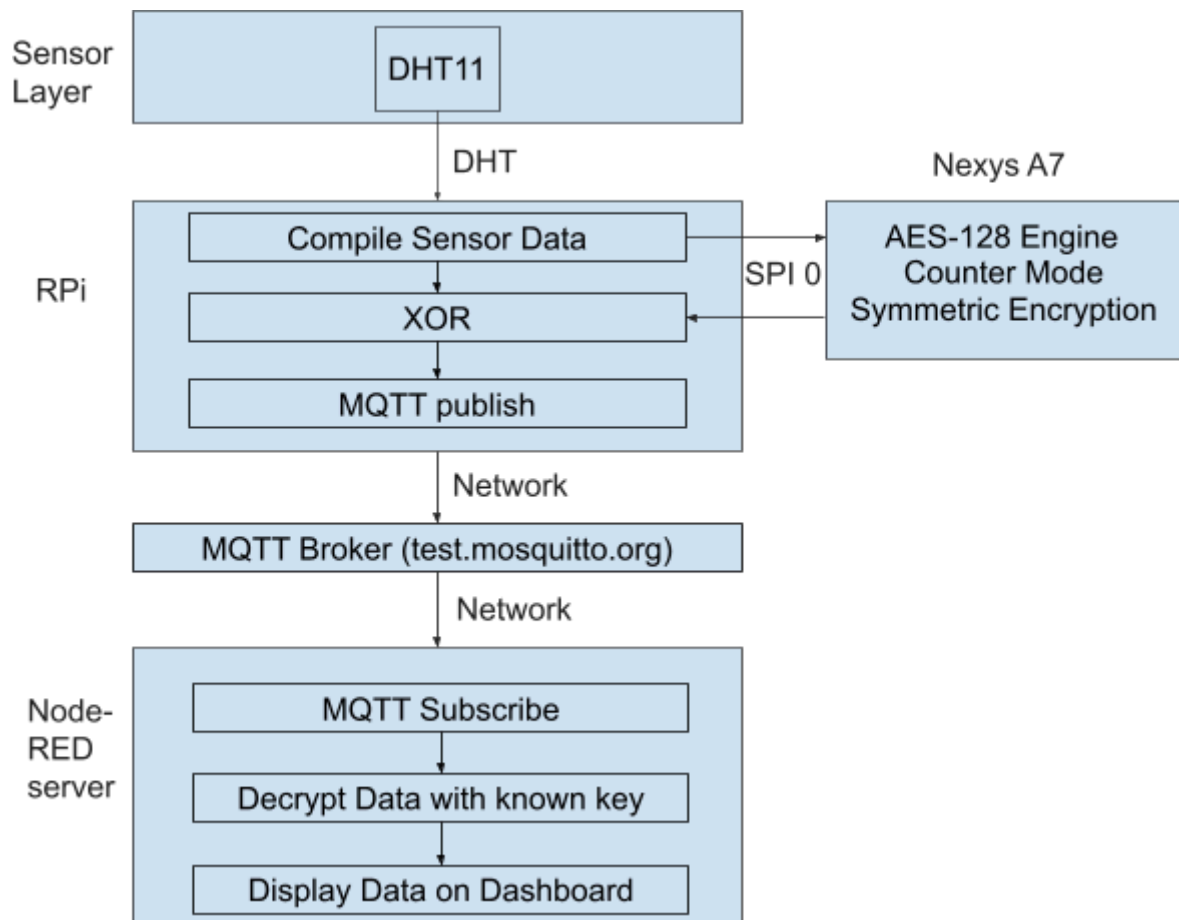


Figure 1 A block diagram of the system.

4. Hardware/Software Used

Hardware	Function	Software	Function
Raspberry Pi 3b	Interface with sensors interface with FPGA Publish to MQTT broker	Vivado ML	Verilog Synthesis PPA Analysis
Nexys A7 FPGA	HDL Deployment	ModelSim	HDL Simulation
Arduino MKR WiFi 1010	SPI verification with FPGA	Node-Red	Design flows and Dashboard UI
DHT-11	Temperature/Humidity Sensor		

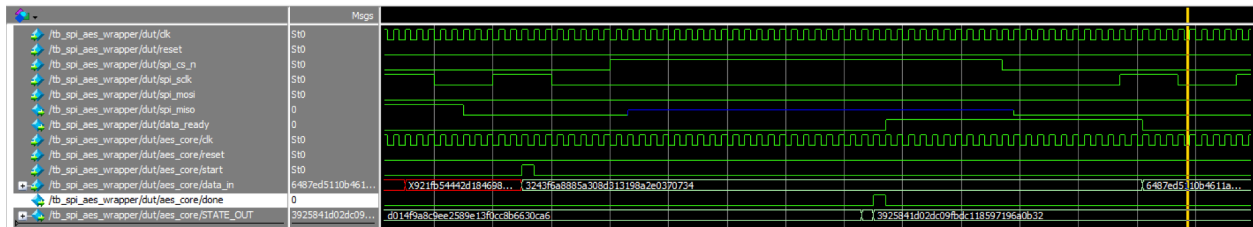
5. Final Results and Demo

Testing was performed throughout the entire development process. Before diving into the hardware implementation of the Advanced Encryption Standard, it was important to both thoroughly understand the cipher as well as ensure that we have a working example or simulation to later reference during our real hardware tests. Therefore, using {cite/reference AES slides here}, a python script was developed in a way to best emulate the behavior of the hardware. This was crucial to verifying our initial AES architecture.

Input : 3243f6a8885a308d313198a2e0370734			
Key : 2b7e151628aed2a6abf7158809cf4f3c			
round[0].input	3243f6a8885a308d313198a2e0370734		
round[0].k_sch	2b7e151628aed2a6abf7158809cf4f3c		
round[1].start	193de3bea0f4e22b9ac68d2ae9f84808		
round[1].s_box	d42711aee0bf98f1b8b45de51e415230		
round[1].s_row	d4bf5d30e0b452aeb8411f11e2798e5		
round[1].m_col	046681e5e0cb199a48f8d37a2806264c		
round[1].k_sch	a0fafe1788542cb123a339392a6c7605		
round[2].start	a49c7ff2689f352b6b5bea43026a5049		
round[2].s_box	49ded28945db96f17f39871a7702533b		
round[2].s_row	49db873b453953897f02d2f177de961a		
round[2].m_col	584dcafc11b4b5aacdbe7caa81b6bb0e5		
round[2].k_sch	f2c295f27a96b9435935807a7359f67f		
round[3].start	6aa8f5f0361dde3ef82d24ad26832469a		
round[3].s_box	ac73cf7befc111df13b5d6b545235ab8		
round[3].s_row	1acc1d6b8efb55a7b1323cfd457211b5		
round[3].m_col	75ee0993200b633353c0cf7ebb25d0dc		
round[3].k_sch	3d804774d716fe3e1e237e446d7a883b		
round[4].start	486c4eee671d9d0d4de3b138d65f58e7		
round[4].s_box	52502f2885a45ed7e311c807f6cf6a94		
round[4].s_row	52a4c89485116a28e3cf2fd7f6505e07		
round[4].m_col	0fdddaa9603138bf6fc0106b5eb31301		
round[4].k_sch	ef44a541a8525b7fb671253db0bad00		
round[5].start	e0927fe8c86363c0d9b1355085b8be01		
round[5].s_box	e14fd29be8fbfbba35c89653976cae7c		
round[5].s_row	e1fb967ce8c8ae9b356cd2ba974fffb53		
round[5].m_col	25d1a9adbd11d168b63a338e4c4cc0b0		
round[5].k_sch	round4d1c6f87c839d87caf2b8bc11f915bc		
round[6].start	f1006f55c1924cef7cc88b325db5d50c		
round[6].s_box	a163a8fc784f29df10e83d234cd503fe		
round[6].s_row	a14f3dfe78e803fc10d5a8df4c632923		
round[6].m_col	4b868d6d2c4a8980339df4e837d218d8		
round[6].k_sch	6d88a37a110b3efddbf98641ca0093fd		
round[7].start	260e2e173d41b77de86472a9fdd28b25		
round[7].s_box	f7ab31f02783a9ff9b4340d354b53d3f		
round[7].s_row	f783403f27433df09bb531ff54aba9d3		
round[7].m_col	1415b5bf461615ec274656d7342ad843		
round[7].k_sch	4e54f70a5f5fc9f384a64fb24a6dc4f		
round[8].start	5a4142b119d9dc1fa3e019657a8c040c		
round[8].s_box	be832cc8d43b86c00ae1d44dda64f2fe		
round[8].s_row	be3bd4fed4e1f2c80a642cc0da33864d		
round[8].m_col	00512fd1b1c889ff54766dcdfalb999a		
round[8].k_sch	ead27321b58dbad2312bf5607f8d292f		
round[9].start	ea835cf0044533d655d98ad8596b0c5		
round[9].s_box	87ec4a8cf26ec3d84d4c46959790e7a6		
round[9].s_row	876e46a6f24ce78c4d904ad897ecc395		
round[9].m_col	473794ed40d4e4a5a3703aa64c9f42bc		
round[9].k_sch	ac7766f319fadcd2128d12941575c006e		
round[10].start	eb40f21e592e38848ba113e71bc342d2		
round[10].s_box	e9098972cb31075f3d327d94af2e2cb5		
round[10].s_row	e9317db5cb322c723d27d94af2e2cb5		
round[10].k_sch	d014f9a8c9ee2589e13f0cc8b6630ca6		
round[10].output	3925841d02dc09fbdcd118597196a0b32		

Testing the HDL implementation was a little different from the initial AES cipher simulation. Once the HDL implementation was complete, a testbench written in SystemVerilog was used to simulate real SPI transfers to best characterize any bugs in our design. HDL simulation followed the following structure

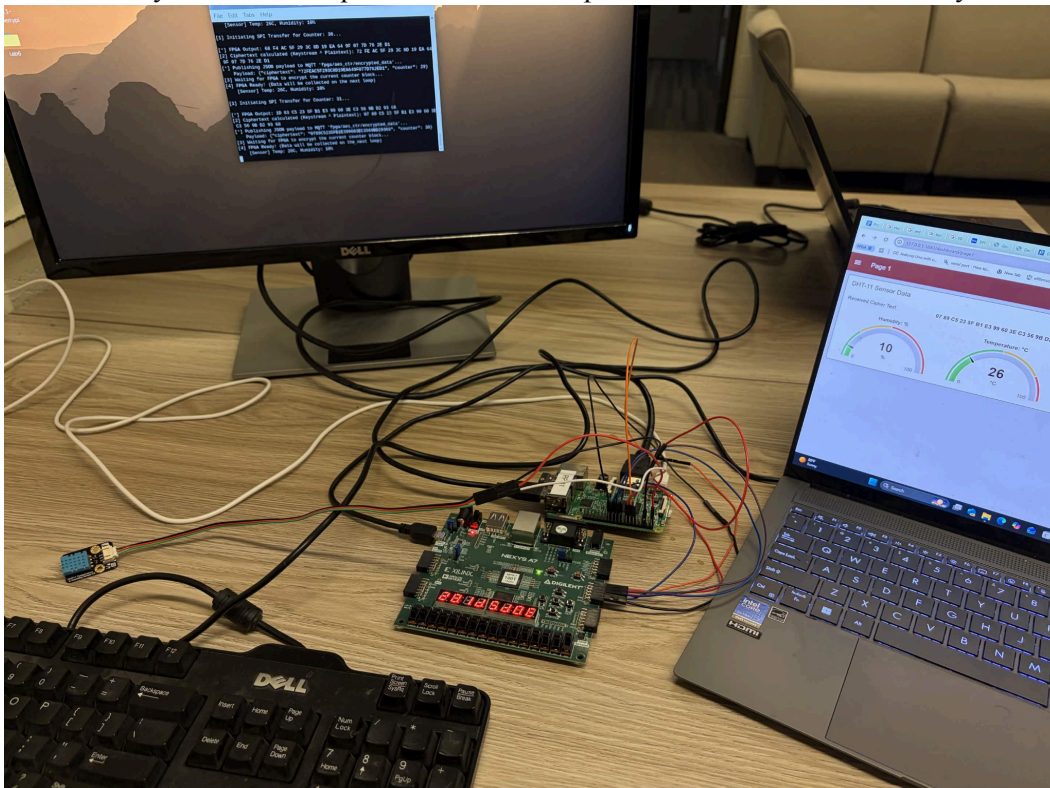
- 1. HDL Code Completed
- 2. Create Testbench to initialize DUT (Make any necessary tweaks to testbench design)
- 3. Simulate using Modelsim (Use Waveform outputs as well as Console Logs to verify functionality)
- 4. If the simulation results weren't up to standard, then make the necessary edits to the HDL implementation and repeat the process



```
# -----
# Starting AES SPI Simulation...
# -----
# 1. Master asserting CS and sending 16 bytes of plaintext...
#   Plaintext transmission complete.
# 2. Waiting for data_ready interrupt from FPGA...
#   Interrupt received! Data is ready.
# 3. Master asserting CS to read ciphertext...
# -----
# Encryption Complete.
# Ciphertext received by Master:
# 39 25 84 1d 02 dc 09 fb dc 11 85 97 19 6a 0b 32
# -----
```

System Level Testing and Verification:

The system as a whole features a Nexys A7 FPGA, Raspberry pi 3b, DHT11 temperature/humidity sensor, and a laptop to host the MQTT broker as well as view the results on the NodeRed Dashboard. Integration of these systems often required the individual pieces to be verified individually.



- SPI Functionality:
 - SPI functionality was tested first using the programmed Nexys A7 FPGA and an Arduino MKR WiFi 1010. This was done do the ease of setting up an Arduino to get work done. Since I didn't have SSH enabled on the PI, doing any work on the PI without access to a monitor was not possible.
 - SPI functionality was deemed working by using the Arduino serial monitor.
 - {arduino mkr wifi image}
- Verification using the .js flow.
 - Once the RPi and Nexys A7 were integrated along a node-red flow and server, it was necessary to perform a system wide verification. This means, comparing the sensor data being read by the Pi against the decrypted data in the node Red dashboard.

Demo Break Down:

```

File Edit Tabs Help
[Sensor] Temp: 26C, Humidity: 10%

[1] Initiating SPI Transfer for Counter: 18...

[*] FPGA Output: B8 43 CC BE 49 8B 53 08 B5 47 FF 4B 97 F1 4A 9E
[2] Ciphertext calculated (Keystream ^ Plaintext): A2 49 CC BE 49 8B 53 08 B5 47
    FF 4B 97 F1 4A 9E
[*] Publishing JSON payload to MQTT 'fpga/aes_ctr/encrypted_data'...
    Payload: {"ciphertext": "A249CCBE498B5308B547FF4B97F14A9E", "counter": 17}
[3] Waiting for FPGA to encrypt the current counter block...
[4] FPGA Ready! (Data will be collected on the next loop)
    [Sensor] Temp: 26C, Humidity: 10%

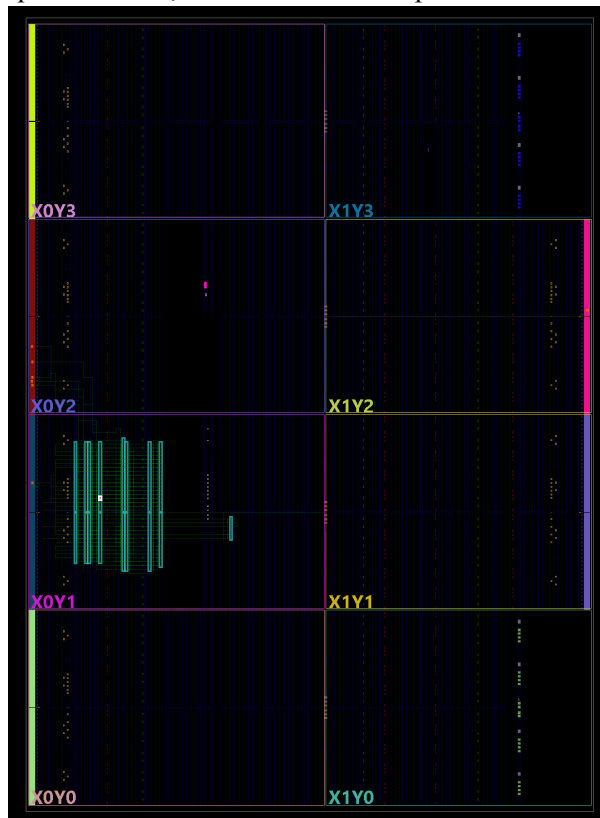
[1] Initiating SPI Transfer for Counter: 19...

[*] FPGA Output: C4 DD A4 44 58 A9 C5 7D 21 B1 0A C8 44 42 C4 02
[2] Ciphertext calculated (Keystream ^ Plaintext): DE D7 A4 44 58 A9 C5 7D 21 B1
    0A C8 44 42 C4 02
[*] Publishing JSON payload to MQTT 'fpga/aes_ctr/encrypted_data'...
    Payload: {"ciphertext": "DED7A44458A9C57D21B10AC84442C402", "counter": 18}
[3] Waiting for FPGA to encrypt the current counter block...
[4] FPGA Ready! (Data will be collected on the next loop)
    [Sensor] Temp: 26C, Humidity: 10%
  
```

- The provided images show our system demo setup. The Nexys A7 Dev and interfacing via the SPI protocol using PMOD ports (JA) and GPIO pins respectively. Additionally, the DHT-11 Sensor is connected to the RPi via its GPIO pins.
- Demo process: (Using Windows PC to initialize node-red)
 - 1. Connect FPGA (PMOD) to RPi (GPIO) and connect DHT-11 Sensor to RPi (GPIO)
 - 2. Open a Powershell Terminal and execute the command: node-red
 - Navigate to your local node-red site using your browser: ex <http://127.0.0.1:1880/>
 - Create a new [flow.js](#) or import an existing [flow.js](#) file.
 - Once ready, Hit Deploy
 - 3. Generate bitstream (.bit file) and program onto FPGA board.
 - 4. Execute spi_mqtt_json_dht11.py on the Pi.
 - 5. Open Dashboard using Node-Red UI

Performance and Benchmarking:

- Throughput, Latency (Core alone)
 - $Throughput_{original} = \frac{128 \text{ bits}}{30 \text{ cycles}} \times 100 \text{ MHz}$
 - $= \frac{128 \text{ bits} \times 100 \times 10^6 \text{ cycles/sec}}{30 \text{ cycles}}$
 - $= 426,666,666.67 \text{ bits/sec}$
 - $= 426.67 \text{ Mbps}$ —OR— 53.33 MB/s
 - 30 Cycle Latency between input to encryption core to cipher text output
- SPI throughput
 - Note: the SPI clock was set to 100kHz for ease of viewing FPGA output during the demo. Design was also demoed using a 500kHz SPI clock.
 - $SPI_{throughput} = 500,000 \text{ cycles/sec} \times 1 \text{ bit/sec}$
 - $SPI_{throughput} = 500,000 \text{ bits/sec}$
 - $SPI_{throughput} = 0.5 \text{ Mbps}$ (SYSTEM BOTTLENECK)
- RPi AES:
 - RPi 5 (dedicated Hardware Acceleration for AES => 6.4 - 7.2 Gbps)
 - RPi 4 (SW Based AES => 400 -640 Mbps)
 - RPi 2.3 (SW based AES => ~10 Mbps)
- Resource Usage, Implementation, and Power Consumption



Power

Total On-Chip Power:

0.117 W

Junction Temperature:

25.5 °C

Thermal Margin:

74.5 °C (16.1 W)

Effective θ JA:

4.6 °C/W

Power supplied to off-chip devices:

0 W

Confidence level:

Low

Implemented Power Report

Utilization

Post-Synthesis

Post-Implementation

Graph

Table

Resource	Utilization	Available	Utilization %
LUT	1607	63400	2.53
FF	922	126800	0.73
IO	7	210	3.33
BUFG	1	32	3.13

- AES Core (Pipelined Version)
 - It is also worthwhile to mention that another version of the core (though not fully integrated into the system) was verified and synthesized. This design features a fully pipelined AES-128 implementation with an initial latency of 10 cycles after which the core is capable of outputting 1 cipher text every cycle.
 - $Throughput_{pipelined} = 128 \text{ bit} \times 100 \text{ MHz}$
 - $= 12,800,000,000 \text{ bits/sec}$
 - $= 12.8 \text{ Gbps}$ —OR— $= 1.6 \text{ GB/s}$

Discussion:

- This first thing to notice is that offloading the AES encryption to an FPGA offers an immense increase in throughput (specifically for the core itself). However, what becomes extremely evident is that using the SPI protocol in this system becomes our bottleneck. A 100 kHz SPI clock was used during the demo. This decreases the full system throughput to .5 Mbps (SPI throughput). This means that using an FPGA paired with this SPI clock offers zero benefit other than theoretical power reduction. However, the SPI clock on the raspberry pi (or a microcontroller like the Arduino MKR WiFi 1010) can support a SPI clock of around 12 MHz up to around 30 MHz. Upping the clock to a frequency in this range bumps the system to a throughput that is equivalent to, if not greater, than the would be throughput if we were to simply offload encryption to the Pi or another microcontroller. This increase in throughput (or maintenance in throughput) in addition to the lower power consumption of the FPGA design, make this a compelling system prototype for edge IoT applications.

6. Team Contribution

Team Member	Task/Deliverable
Hayden Fuller	Sensor interfacing and data compilation
Edgar Muniz	Development of AES-128 Python Golden Model
Edgar Muniz	Development of SystemVerilog implementation of AES-128 + FPGA Synthesis for Nexys A7
Edgar Muniz	Interfacing between RPi and Nexys A7: Development of SPI HDL code
Edgar Muniz - Hayden Fuller	Interfacing between RPi and Nexys A7: Development of Python Code
Hayden Fuller	MQTT broker setup and publishing
Hayden Fuller - Edgar Muniz	NodeRed Dashboard
Edgar Muniz - Hayden Fuller	Node.js flow
Edgar Muniz - Hayden Fuller	Final Report

7. Conclusion

In summary, through this project, the team explored the application of hardware acceleration for edge IoT applications. While this specific prototype utilized a Raspberry Pi 3B to execute processor functionality such as sensor data collection, using a dedicated MCU would further validate the project's findings. The power utilization of the FPGA Dev board (~.110 W) combined with many low power modes for other microcontrollers as well as the impressive throughput performance demonstrates that this device successfully meets the group's initial objective.

8. References

<https://www.sciencedirect.com/science/article/pii/S2542660525000678>

P. Cíbk, S. Ricci, Patrik Dobiáš, J. Hajný, L. Malina, and J. Havlín, "Quantum-resistant hardware-accelerated IoT traffic encryptor," *Internet of Things*, pp. 101554–101554, Mar. 2025, doi: <https://doi.org/10.1016/j.iot.2025.101554>.

<https://ieeexplore.ieee.org/abstract/document/8536371>

C. -C. Chang, W. -K. Lee, Y. Liu, B. -M. Goi and R. C. . -W. Phan, "Signature Gateway: Offloading Signature Generation to IoT Gateway Accelerated by GPU," in *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4448-4461, June 2019, doi: 10.1109/JIOT.2018.2881425.